Networking: HTTP, JSON, and code generation

Talking to a server from Flutter, without hand-writing JSON

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

Speak HTTP correctly

Methods, safe vs idempotent, and the status codes and headers that change what your app does next.

Survive a flaky network

dio's failure paths, interceptors, cancel tokens, and retrying with backoff and jitter.

Stop hand-writing JSON

json_serializable and build_runner generate the mapping code; freezed generates the model itself.

Wrap the API in a repository

One class that talks to dio and JSON, so the bloc from Lecture 6 never sees either.

PART 1 · HTTP

Talking to a server, correctly

Methods, status codes, headers and timeouts

A request, and the response it earns

Every HTTP call is a **verb** and a **path**, a few **headers**, and an optional **body**, answered by a **status code** and a body.

A mobile round trip costs roughly 40–200 ms on a good connection, seconds on a bad one, and nothing at all inside a tunnel.

The client builds the request and reads the answer. What runs behind that URL is next semester's material.

Everything in this lecture is the client half of that line.

One round trip is expensive on mobile. Timeouts, retries and generated JSON all exist because that round trip can be slow or simply fail.

Safe is not the same as idempotent

Safe: does not modify state. **Idempotent**: sending it twice has the same effect as once.

METHOD	SAFE	IDEMP.	WHAT YOU USE IT FOR
GET	yes	yes	read a resource
HEAD	yes	yes	headers only: size and caching
PUT	no	yes	replace a resource at a known URL
DELETE	no	yes	remove it; repeating leaves it gone
POST	no	no	create, or anything else
PATCH	no	no	partial update, not idempotent

Every safe method is idempotent; the reverse is false. PUT and DELETE modify state, but repeating them does not change it further. That is what makes them safe to retry, and POST not.

Status codes that matter

CODE	WHAT THE CLIENT DOES ABOUT IT
200 / 201 / 204	success; 201 after a create, 204 after a delete with no body
301 / 302	follow the new location; do not resend a POST body on 301
400 / 422	the request is malformed; fix it, do not retry
401 / 403	not authenticated or not allowed; refresh the token or stop
404	the resource is gone; show that, do not retry
429	rate limited; back off and retry after the given delay
500 / 502 / 503 / 504	the server failed; safe to retry with backoff

The full range is 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error.

Headers and timeouts

Headers worth knowing

`Content-Type`: what the body is, almost always `application/json`.

`Authorization`: `Bearer <token>` on every authenticated call.

`Accept`: what format the client wants back.

`Idempotency-Key`: a client-generated id so a retried POST is not applied twice.

```
final dio = Dio(BaseOptions(
  connectTimeout:
      const Duration(seconds: 5),
  receiveTimeout:
      const Duration(seconds: 10),
));
```

The default timeout on mobile is effectively none: a request can hang until the user force-closes the app.

NUMBERS

Why a timeout budget matters

40–200 ms

round trip on a good mobile connection

5 s

a reasonable connect timeout to start from

0

the default timeout with nothing set

PART 2 · DIO AND HTTP

The client side of the wire

Requests, failure paths, interceptors and the minimal alternative

dio: a request, and its failure paths

```dart
final dio = Dio(BaseOptions(
    baseUrl: 'https://api.example.com'));
Future<Reading> fetchReading(String id)
    async {
  try {
    final r =
        await dio.get('/readings/$id');
    return Reading.fromJson(r.data);
  } on DioException catch (e) {
    throw ApiFailure(
        e.message ?? 'network error');
  }
}
```

http or dio?

`package:http` is minimal: one function per request.

`dio` adds base URLs, timeouts, interceptors, cancellation.

Map `DioException` to your own error type at the edge.

Interceptors: one place for shared behavior

```
dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) {
    options.headers['Authorization'] =
        'Bearer $token';
    handler.next(options);
  },
  onError: (e, handler) {
    log('ERROR ${e.response?.statusCode}');
    handler.next(e);
  },
));
```

Why interceptors exist

Attach the auth header to every call in one place, not at every call site.

Log requests and responses without touching business code.

Chain them: auth, then logging, then retry.

Cancel tokens: stop a request in flight

```
CancelToken? _token;

Future<void> search(String query) async {
  _token?.cancel('superseded');
  _token = CancelToken();
  final res = await dio.get('/search',
      queryParameters: {'q': query},
      cancelToken: _token);
  render(res.data);
}
```

Search as you type

Every keystroke fires a new request. Cancel the previous one before sending the next.

A canceled request throws `DioExceptionType.cancel`, which you simply ignore.

Also cancel on `dispose()`, so a closed screen does not update dead state.

package:http, the minimal alternative

```dart
final res = await http.get(
    Uri.parse('$base/readings/$id'));

if (res.statusCode == 200) {
  return Reading.fromJson(
      jsonDecode(res.body));
}
throw ApiFailure('status
    ${res.statusCode}');
```

One function per request

No client object, no interceptors, no cancel tokens: check the status code yourself.

`RetryClient` from the same package wraps retries around any request.

Fine for a small app or a script. dio earns its weight once auth headers go on every call.

PART 3 · RESILIENCE

When the network says no

Retry with backoff and jitter, and when not to retry

Retry with exponential backoff and jitter

```
Future<T> withRetry<T>(
    Future<T> Function() call) async {
  for (var a = 0; ; a++) { try {
      return await call();
    } on DioException catch (e) {
      if (!_retryable(e) || a == 4) rethrow;
      final wait =
          Random().nextInt(200 << a);
      await Future.delayed(
          Duration(milliseconds: wait));
    }
  }
}
```

Why jitter

Doubling alone (200, 400, 800 ms) still lines up every client on the same millisecond.

The wait is a random draw from the interval, so retries spread out.

RESILIENCE

Which failures are worth retrying

OUTCOME	WHAT TO DO
408, 429, 5xx	transient, retry with backoff
connection reset, no response	the connection failed, retry with backoff
400, 422	client error, fix the request, never retry
401, 403	refresh the token or stop, do not blindly retry
404	the resource is gone, do not retry
POST, no idempotency key	never retry; a repeat may create it twice

Retrying is a decision, not a default. Wrap only calls where a repeat is provably harmless: safe methods, plus POST with an idempotency key.

Thundering herds and hedged requests

When a service fails briefly, thousands of clients fail at once. If every one retries after the same delay, the wave is larger than the original load: a thundering herd.
Full jitter, from the previous slide, spreads those retries across the interval, so the recovering server sees load ramp up instead of arriving all at once.
Hedging is the opposite problem. A few requests are simply slow. Send a second copy after a short delay, take whichever answers first, cancel the other.

ATTEMPT	SENT AT	OUTCOME
1	t = 0 ms	silent at 250 ms, canceled
2, the hedge	t = 250 ms	answers 60 ms later, used

Retries fix failures. Hedges fix slowness. Both multiply load: cap it to one hedge, only past the slow tail, only for safe or idempotent calls.

PART 4 · JSON

Stop hand-writing the mapping

Generated fromJson and toJson, and freezed

JSON in Dart, the manual way

```dart
class Reading {
  final String deviceId; final double tempC;
  Reading({required this.deviceId,
      required this.tempC});
  factory Reading.fromJson(Map j) =>
    Reading(
    deviceId: j['device_id'] as String,
    tempC: (j['temp_c'] as num).toDouble(),
  );
  Map toJson() => {
    'device_id': deviceId, 'temp_c': tempC,
  };
}
```

Where this breaks

A renamed field silently becomes `null` at runtime, not at compile time.

Every model needs this twice, kept in sync by hand.

Nested objects multiply the boilerplate fast.

@JsonSerializable and the generated half

```dart
part 'reading.g.dart';
@JsonSerializable()
class Reading {
  const Reading({required this.deviceId,
      required this.tempC});
  @JsonKey(name: 'device_id')
  final String deviceId;
  @JsonKey(name: 'temp_c')
  final double tempC;
  factory Reading.fromJson(Map j) =>
      _$ReadingFromJson(j);
  Map toJson() => _$ReadingToJson(this);
}
```

```
dart run build_runner build \
  --delete-conflicting-outputs
```

`part` links to the generated twin. Never edit a `.g.dart` file.

The generator fails the build instead of returning a silent null.

Commit `.g.dart` files, or generate them in CI.

Enums, dates and nested objects

```
enum Status { pending, done }
@JsonSerializable(explicitToJson: true)
class Task {
  final Status status;
  @JsonKey(name: 'due_at')
  final DateTime dueAt;
  final Reading lastReading;
  Task({required this.status,
      required this.dueAt,
      required this.lastReading});
  factory Task.fromJson(Map j) =>
    _$TaskFromJson(j);
}
```

Enums serialize by name; `@JsonKey` still renames the wire field.

`DateTime` round-trips through ISO-8601 with no extra code.

`explicitToJson: true` calls a nested field's own `toJson`, not `toString`.

JSON

The generators you will actually add

PACKAGE	GENERATES	WHY ADD IT
json_serializable	fromJson / toJson	every project with an API has this one
freezed	immutable classes, sealed unions	loading / data / error states, next slide
retrofit	a typed dio client	one place where endpoints are declared
riverpod_generator	providers from functions	typos become build errors
drift	type-checked SQL	storage that survives offline
mockito	mocks for your interfaces	tests that skip the network

build_runner is slow on large projects and fails when generated files go stale. It still belongs in CI.

freezed: the model writes itself

```dart
part 'task_state.freezed.dart';

@freezed
class TaskState with _$TaskState {
  const factory TaskState.loading() =
    Loading;
  const factory TaskState.data(
      List<Task> tasks) = Data;
  const factory TaskState.error(
      String message) = Error;
}
```

One annotation generates `copyWith`, `==`, `hashCode`, and `toString`, for every case.

The three constructors form a sealed union: a `switch` over the state must handle all of them or the compiler complains.

This is the state shape Lecture 6 builds a Cubit around.

Which JSON approach fits

APPROACH	COST	USE WHEN
Hand-written	no dependency, full control	one tiny model, or a one-off script
json_serializable	one annotation, a build step	any model that talks to an API
freezed	json_serializable plus copyWith and unions	state classes and models that need equality

freezed can generate fromJson and toJson itself, by adding json_serializable alongside it: the two packages compose.

PART 5 · ARCHITECTURE

Where the network code lives

A repository between the API and the bloc, and the secrets rule

A repository class wraps the API

```
class TaskRepository {
  TaskRepository(this._dio); final Dio _dio;
  Future<List<Task>> fetchTasks() async {
    final res = await withRetry(() => _dio.get('/tasks'));
    return (res.data as List).map(Task.fromJson).toList();
  }
  Future<Task> addTask(Task task) async {
    final res = await _dio.post('/tasks', data: task.toJson());
    return Task.fromJson(res.data);
  }
}
```

The bloc from Lecture 6 calls `fetchTasks()` and `addTask()`, never Dio or a JSON key directly.

ARCHITECTURE

Why the layer earns its keep

LAYER	TALKS TO
Widget	the Bloc, through `context.read` and `BlocBuilder`
Bloc / Cubit	the Repository's methods, never Dio or a JSON key directly
Repository	Dio, retry, and the model classes; the only layer that knows the API shape

The repository is what you mock, not the network. A fake Repository in a test returns fixed Task objects instantly. Nothing else in the app changes to make that possible.

ARCHITECTURE

API keys never live in the app

An APK can be unpacked and a release binary can be inspected. Anything shipped inside the app, including a key passed through `--dart-define`, can be extracted.

A key that must stay secret belongs on the server. The app calls your endpoint; your endpoint calls the third-party API and adds the key.

If the key must be reachable from the client anyway, a public rate-limited key, treat leaking it as expected and restrict it by package name or domain instead.

Cloud AI keys are the case students hit most often. Lecture 12 covers the correct shape end to end.

The client is not a safe place for a secret. This applies to every key, token, or credential, not only the ones for AI APIs.

Putting it together: adding a task

1. The UI calls `context.read<TaskCubit>().add(task)`.
2. The Cubit calls `repository.addTask(task)` and emits a loading state.
3. The Repository serializes `task.toJson()`, sends it through dio wrapped in `withRetry`, and parses the response with `Task.fromJson`.
4. A failure becomes an `ApiFailure`, the Cubit emits an error state, and `BlocBuilder` shows it. No layer above the Repository ever saw Dio.

Every topic from today lives inside one method. Status codes, retry with jitter, and generated JSON all sit inside the Repository. Every layer above it stays free of network detail.

RECAP

Three things to remember

1. **Safe and idempotent are different properties.** Only safe or idempotent calls, or a POST with an idempotency key, are safe to retry.
2. **Retry with jitter, not a fixed backoff.** A thundering herd is a synchronized retry wave, and jitter is what breaks the synchronization.
3. **Generate the JSON mapping, do not hand-write it.** json_serializable catches a renamed field at build time; a hand-written cast fails silently at runtime.

FURTHER READING

pub.dev/packages/dio · pub.dev/packages/json_serializable · pub.dev/packages/freezed

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel